

Semestrální práce

Smart Food Delivery Platform

NSS

letní semestr B252

Smart Food Delivery Platform

1. Popis aplikace a Motivace

Popis: Smart Food Delivery Platform je webová aplikace pro správu a koordinaci doručování jídla od restaurací k zákazníkům. Systém propojuje zákazníky, restaurace, kurýry a administrátora. Zákazník může vyhledat restauraci, vytvořit objednávku a sledovat její stav. Restaurace spravuje jídelní lístek a potvrzuje objednávky. Kurýr přijímá rozvoz a aktualizuje stav doručení. Administrátor spravuje uživatele a restaurace.

Motivace:

Cílem systému je zjednodušit komunikaci mezi zákazníkem, restaurací a kurýrem při doručování jídla. Bez jednotného systému by bylo nutné objednávky, potvrzení restaurací a stav doručení řešit odděleně, což zvyšuje riziko chyb a zdržení.

Systém řeší:

- vytvoření a správu objednávek,
- potvrzení objednávky restaurací,
- přijetí nebo odmítnutí rozvozu kurýrem,
- sledování stavu objednávky zákazníkem,
- správu jídelního lístku restaurací,
- základní správu uživatelů administrátorem.

2. Funkční požadavky

ID	Požadavek	Priorita
FR-01	Zákazník se může registrovat a přihlásit (email + heslo)	Vysoká
FR-02	Zákazník může vyhledat restaurace a jídla dle názvu nebo kategorie	Vysoká
FR-03	Zákazník může vytvořit objednávku z jídelního lístku restaurace	Vysoká
FR-04	Zákazník může sledovat stav své objednávky	Vysoká
FR-05	Systém přiřadí objednávku kurýrovi	Vysoká
FR-06	Kurýr může zobrazit přiřazené objednávky a aktualizovat stav doručení	Vysoká
FR-07	Restaurace může spravovat jídelní lístek	Střední
FR-08	Systém odesílá notifikace o změně stavu objednávky	Střední
FR-09	Zákazník může hodnotit restauraci a kurýra	Střední
FR-10	Admin může spravovat uživatele a restaurace	Nízká

3. Nefunkční požadavky

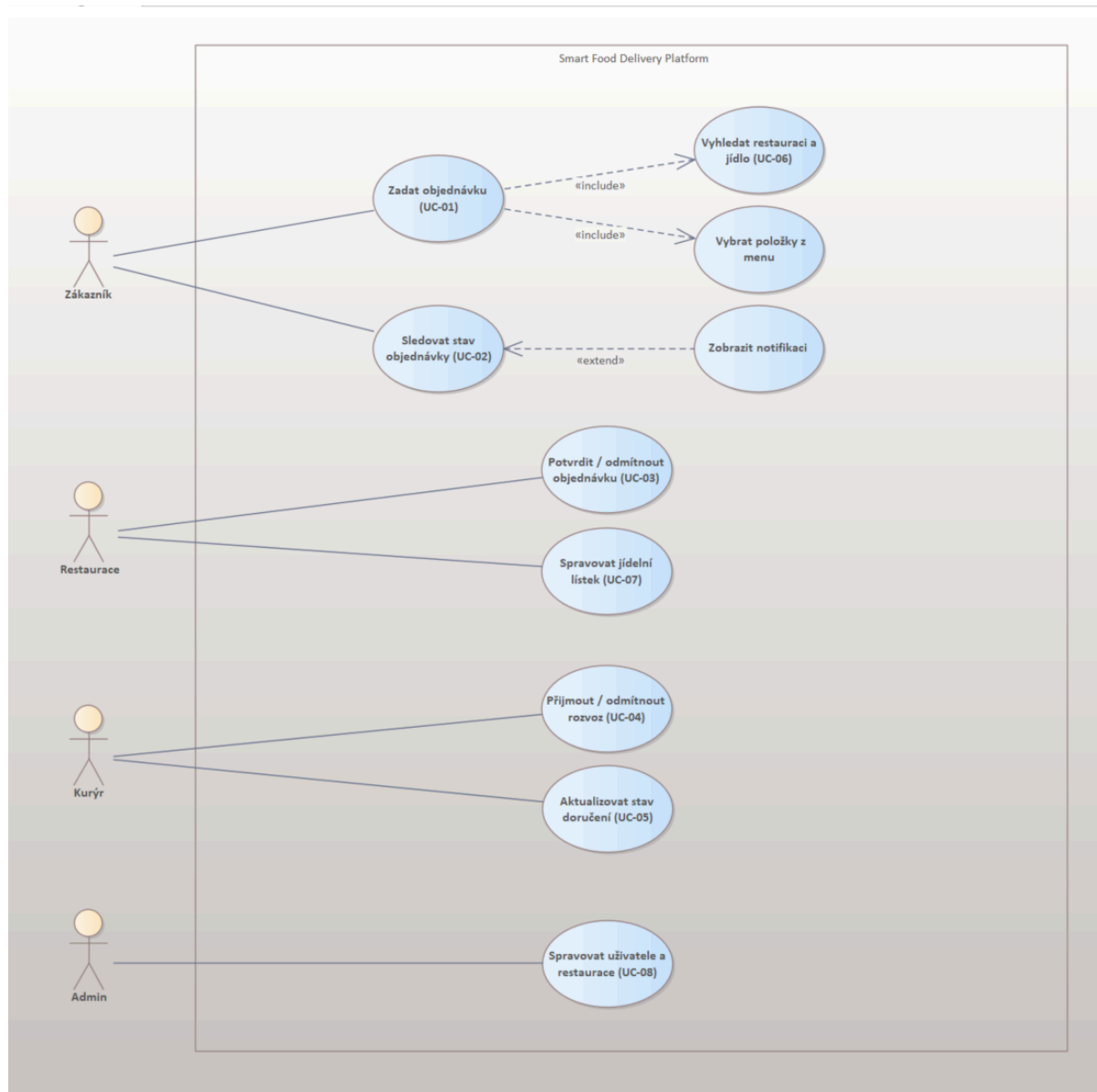
ID	Požadavek	Metrika
NFR-01	Aplikace bude dostupná jako webová aplikace přes HTTP	Funkční přístup z klientské aplikace
NFR-02	Systém bude používat základní autentizaci (Basic Auth) pro chráněné operace	100 % chráněných endpointů
NFR-03	Systém bude ukládat data do relační databáze	Perzistence dat pro hlavní entity systému
NFR-04	Systém bude implementován v jazyce Java pomocí Spring Boot	Použitá technologie
NFR-05	Služby budou komunikovat synchronně pomocí REST API	HTTP odpovědi 200 / 201 / 400 / 401
NFR-06	Systém bude využívat Kafka pro asynchronní komunikaci mezi službami	Zpracování vybraných událostí objednávky
NFR-07	Systém bude rozdělen na více samostatných služeb podle doménových oblastí	Oddělené služby CatalogService, OrderService, DeliveryService
NFR-08	Systém bude validovat vstupy uživatele a vracet odpovídající chybové odpovědi	Nevalidní vstup vrací 400 Bad Request
NFR-09	Systém bude využívat cache pro často čtená data	Hazelcast cache pro katalog nebo stav objednávky

4. Seznam uživatelů (aktérů)

Akteur	Popis	Oprávnění
Zákazník	Uživatel, který vyhledává restaurace a jídla, vytváří objednávky a sleduje jejich stav	Vytvoření objednávky, sledování objednávky, vyhledávání restaurací a jídel
Restaurace	Uživatel zastupující restauraci, která spravuje svůj jídelní lístek a reaguje na nové objednávky	Správa jídelního lístku, potvrzení nebo odmítnutí objednávky
Kurýr	Uživatel, který zajišťuje doručení objednávky zákazníkovi.	Přijetí nebo odmítnutí rozvozu, aktualizace stavu doručení
Admin	Správce systému, který má oprávnění spravovat uživatele a restaurace.	Správa uživatelů a restaurace

5. Případy užití (Use Cases)

ID	Název	Aktér	Popis
UC-01	Zadat objednávku	Zákazník	Zákazník vybere restauraci, prohlédne si nabídku jídel a přidá vybrané položky do objednávky. Následně objednávku potvrdí a systém ji předá restauraci ke zpracování.
UC-02	Sledovat stav objednávky	Zákazník	Zákazník si zobrazí detail své objednávky a sleduje její aktuální stav, například vytvořena, potvrzena, připravována, doručována nebo doručena.
UC-03	Potvrdit / odmítnout objednávku	Restaurace	Restaurace obdrží novou objednávku, zkontroluje její obsah a rozhodne, zda ji přijme nebo odmítne.
UC-04	Přijmout / odmítnout rozvoz	Kurýr	Kurýr obdrží informaci o objednavce určené k doručení a rozhodne se, zda rozvoz přijme nebo odmítne.
UC-05	Aktualizovat stav doručení	Kurýr	Kurýr během doručení aktualizuje stav objednávky, vyzvednuto nebo doručeno.
UC-06	Vyhledat restauraci a jídlo	Zákazník	Zákazník vyhledává restaurace a jídla podle názvu nebo kategorie a vybírá si z dostupné nabídky.
UC-07	Spravovat jídelní lístek	Restaurace	Restaurace spravuje svůj jídelní lístek, přidává nové položky, upravuje existující položky nebo odstraňuje nedostupná jídla.
UC-08	Spravovat uživatele a restaurace	Admin	Administrátor spravuje uživatelské účty a restaurace, vytváří nové záznamy, upravuje existující nebo je odstraňuje.



Obr. č. 1 – Případy užití

6. Class Diagram

Centrální entitou je objednávka (**Order**), která propojuje zákazníka, restauraci, položky objednávky a případně přiřazeného kurýra.

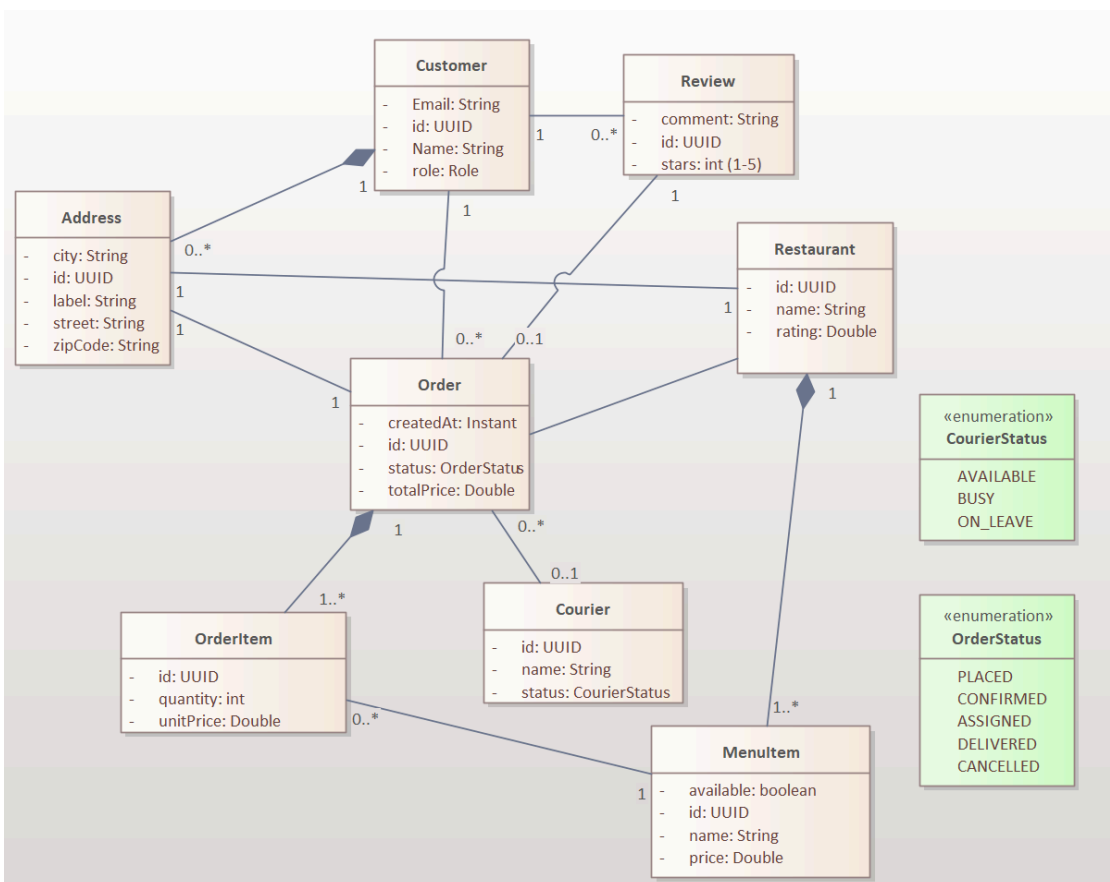
Zákazník (**Customer**) může mít uložených více adres (**Address**) a může vytvářet více objednávek. Každá objednávka patří právě jednomu zákazníkovi a jedné restauraci (**Restaurant**). Restaurace zároveň spravuje svůj jídelní lístek, který je reprezentován entitou Menuitem. Každá položka menu náleží právě jedné restauraci.

Objednávka se skládá z jedné nebo více položek (**OrderItem**), které reprezentují konkrétní vybraná jídla a jejich množství. Každá položka objednávky odkazuje na konkrétní položku z jídelního lístku.

Kurýr (**Courier**) může být přiřazen k objednávce, přičemž jedna objednávka má nejvýše jednoho kurýra, ale kurýr může zpracovávat více objednávek v čase. Stav objednávky je reprezentován výčtovým typem **OrderStatus** (**PLACED, CONFIRMED, ASSIGNED, DELIVERED, CANCELLED**), který umožňuje sledovat její životní cyklus. Stav kurýra je reprezentován výčtem CourierStatus (**AVAILABLE, BUSY, ON_LEAVE**).

Diagram dále obsahuje entitu **Review**, která umožňuje zákazníkovi hodnotit dokončenou objednávku. Hodnocení je navázáno na konkrétní objednávku, protože objednávka již obsahuje vazbu na zákazníka, restauraci i doručení.

Celý model je navržen tak, aby pokrýval vytváření objednávek, správu menu, přiřazení kurýra a sledování průběhu doručení.



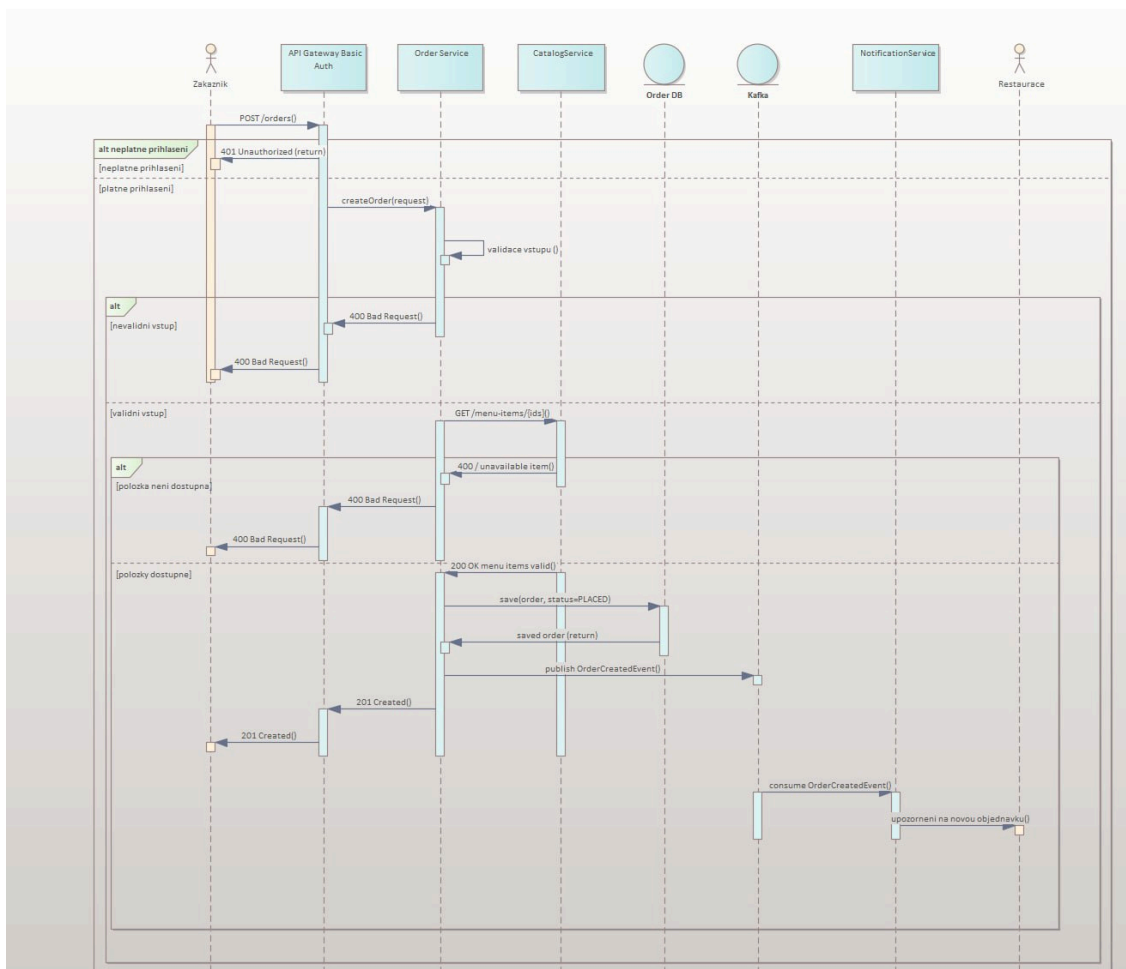
Obr. č. 2 – Class Diagram

7. Sequence diagram

Zákazník odešle požadavek na vytvoření objednávky pomocí **HTTP (POST /orders)**. Tento požadavek je nejprve zpracován API Gateway, kde proběhne ověření přihlašovacích údajů pomocí Basic Auth. Po úspěšném ověření je požadavek předán službě OrderService.

OrderService provede validaci vstupních dat objednávky. Následně si ověří dostupnost vybraných položek pomocí volání služby **CatalogService**. Pokud jsou položky dostupné, objednávka je uložena do databáze služby **OrderService**. Pokud jsou data validní, objednávka je uložena do databáze a zákazník obdrží odpověď **201 Created**. V případě chyby je vrácen odpovídající chybový kód (**400 Bad Request**).

Po vytvoření objednávky je publikována událost do Kafka, která je asynchronně zpracována **NotificationService** pro informování restaurace.



Obr. č. 3 – Sequence diagram, UC-01 Zadání objednávky

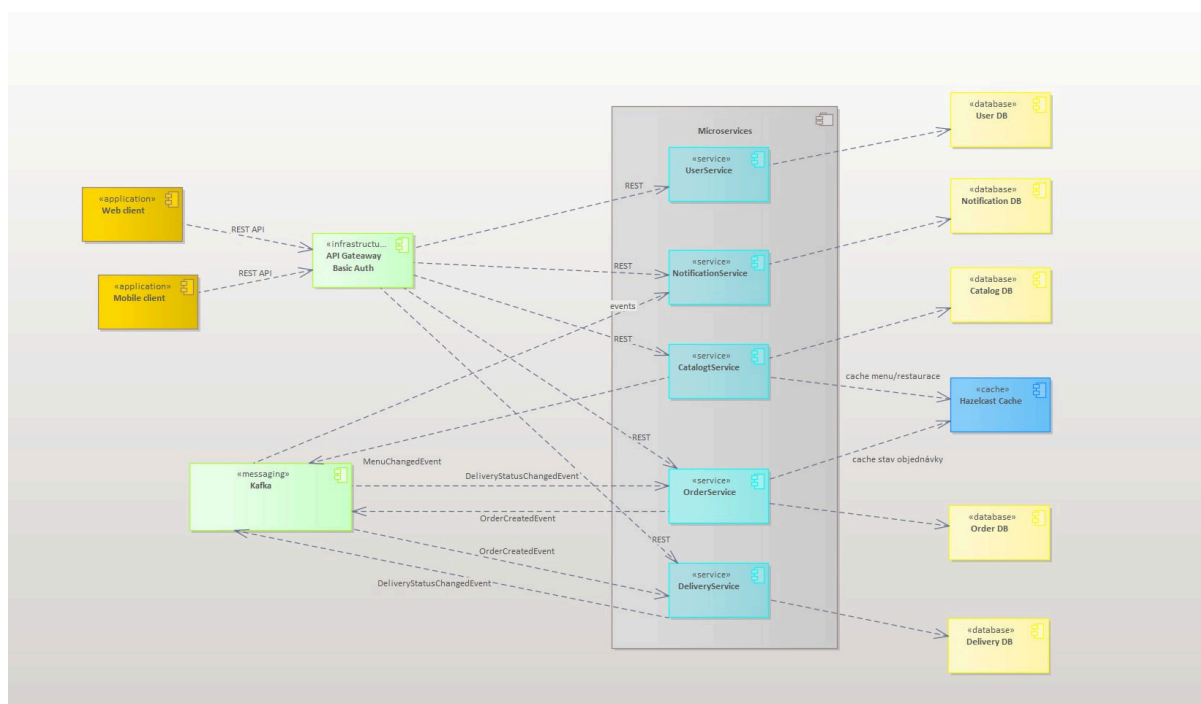
8. Component diagram

Uživatelé přistupují k systému prostřednictvím dvou typů klientů – webového a mobilního. Všechny požadavky směřují přes API Gateway, která zajišťuje jednotný vstup do systému a ověření uživatele pomocí Basic Auth.

Systém je rozdělen na několik hlavních služeb. UserService spravuje uživatele a jejich oprávnění. CatalogService zajišťuje správu restaurací a jídelního lístku. OrderService zpracovává objednávky a jejich stav. DeliveryService se stará o kurýry a doručení. NotificationService zpracovává notifikace.

Každá služba má vlastní datové úložiště, což zabraňuje sdílení databáze mezi službami a snižuje vzájemnou závislost. Pro často čtená data (seznam restaurací nebo stav objednávky), je využita cache (Hazelcast).

Služby komunikují synchronně pomocí REST API v případech, kdy je potřeba okamžitá odpověď (vytvoření objednávky klientem). Pro asynchronní komunikaci je využit Kafka messaging, který slouží k přenosu událostí mezi službami, například při změně stavu doručení. Tyto události jsou následně zpracovány dalšími službami.



Obr. č. 4 – Component Diagram

9. Vyber vhodne architektury

Systém **Smart Food Delivery Platform** je navržen jako **event-driven microservices architektura**. Aplikace je rozdělena do samostatných služeb podle hlavních doménových oblastí: uživatelé, katalog restaurací a jídel, objednávky, doručení a notifikace.

Jednotlivé služby komunikují synchronně pomocí **REST API** v případech, kdy je potřeba okamžitá odpověď klientovi, například při odeslání požadavku na vytvoření objednávky nebo při načtení jídelního lístku. Po úspěšném vytvoření objednávky systém publikuje událost do **Kafka**. Tuto událost následně asynchronně zpracují další služby, například NotificationService pro informování restaurace a DeliveryService pro zahájení procesu přiřazení kurýra.

Každá služba má vlastní datové úložiště, čímž se snižuje vzájemná závislost mezi službami. Pro často čtená data (seznam jídel/restaurací, stav objednávky), je využita **Hazelcast cache**. Každá microservice má interně vrstvenou strukturu (Controller/Service/Repository).

Přehled architektonických rozhodnutí

Rozhodnutí	Důvod	Technologie
Microservices architektura	Oddělení hlavních částí systému podle domény a možnost samostatného rozvoje služeb	Spring Boot microservices
Event-driven komunikace	Události jako vytvoření objednávky nebo změna stavu doručení mohou být zpracovány asynchronně	Kafka
REST API	Synchronní komunikace tam, kde je potřeba okamžitá odpověď klientovi	HTTP / REST
API Gateway	Jednotný vstupní bod do systému a směrování požadavků na konkrétní služby	Spring Cloud Gateway / vlastní gateway
Relační databáze	Strukturované uložení dat jednotlivých služeb	PostgreSQL
Databáze na službu	Každá služba vlastní svá data a nezasahuje přímo do databáze jiné služby	Oddělené DB / schéma
Cache	Snížení počtu opakovaných dotazů pro často čtená data	Hazelcast
Zabezpečení	Ochrana chráněných endpointů	Basic Auth / Spring Security
Vrstvená struktura uvnitř služby	Přehledné oddělení controlleru, business logiky a datové vrstvy	Controller / Service/Repository

Design Patterns

Pattern	Kde je použit	Účel
Repository	OrderRepository, UserRepository, RestaurantRepository	Oddělení přístupu k databázi od business logiky
Service Layer	OrderService, DeliveryService, RestaurantService, CatalogService	Zapouzdření aplikační logiky jednotlivých služeb
State	OrderStatus (PLACED, CONFIRMED, ASSIGNED, DELIVERED, CANCELLED)	Řízení životního cyklu objednávky a povolených přechodů mezi stavy
Strategy	Výběr kurýra (první dostupný kurýr)	Možnost změny algoritmu bez úpravy kódu
Observer / Publish-Subscribe	Kafka eventy mezi službami, např. OrderCreatedEvent, DeliveryStatusChangedEvent	Asynchronní reakce služeb na události bez přímé závislosti mezi nimi
Builder	Vytváření objektu Order z požadavku	Jednodušší konstrukce složitých objektů